

Autoencoders (AEs) and Variational Autoencoders (VAEs)

Unsupervised Representation Learning Models

Previous session

- Architecture of Autoencoder
- Reconstruction Loss Functions (MSE, Binary Cross-Entropy)

Today's Session Overview

Types of Autoencoders: Architecture and Code-Level Understanding

- Shallow Autoencoder
- Deep Autoencoder
- CNN Autoencoder
- Denoising Autoencoder

Limitations of Autoencoders

Shallow Autoencoder (AE)

A **Shallow Autoencoder** is the simplest form of an autoencoder, characterized by:

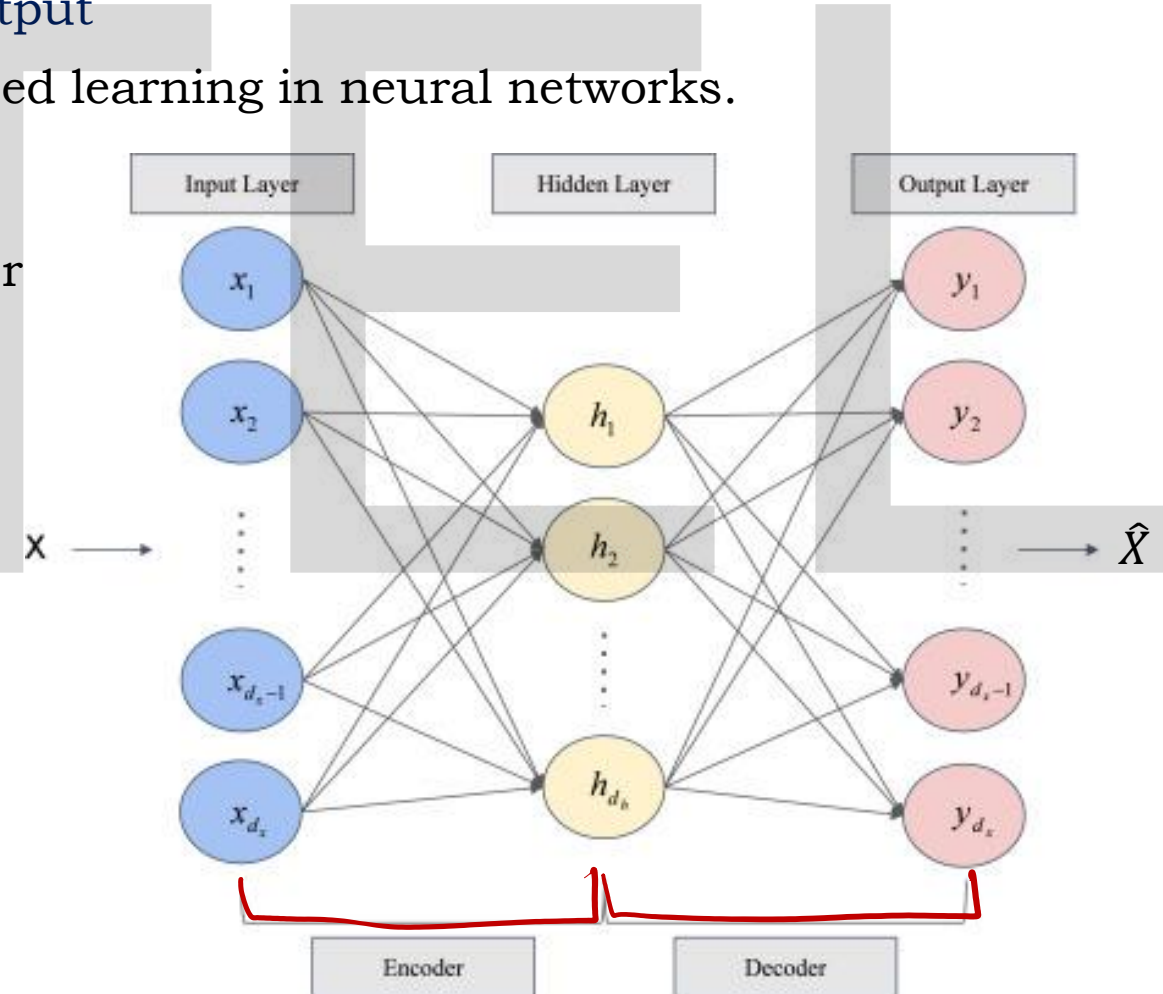
- **Only one hidden layer** between the input and the output

It was the **first step** that opened the door to unsupervised learning in neural networks.

Architecture:

Input Layer $\rightarrow$ Hidden Layer (Latent) $\rightarrow$ Output Layer
 $X \in \mathbb{R}^n$ $z \in \mathbb{R}^m (m < n)$ $\hat{X} \in \mathbb{R}^n$

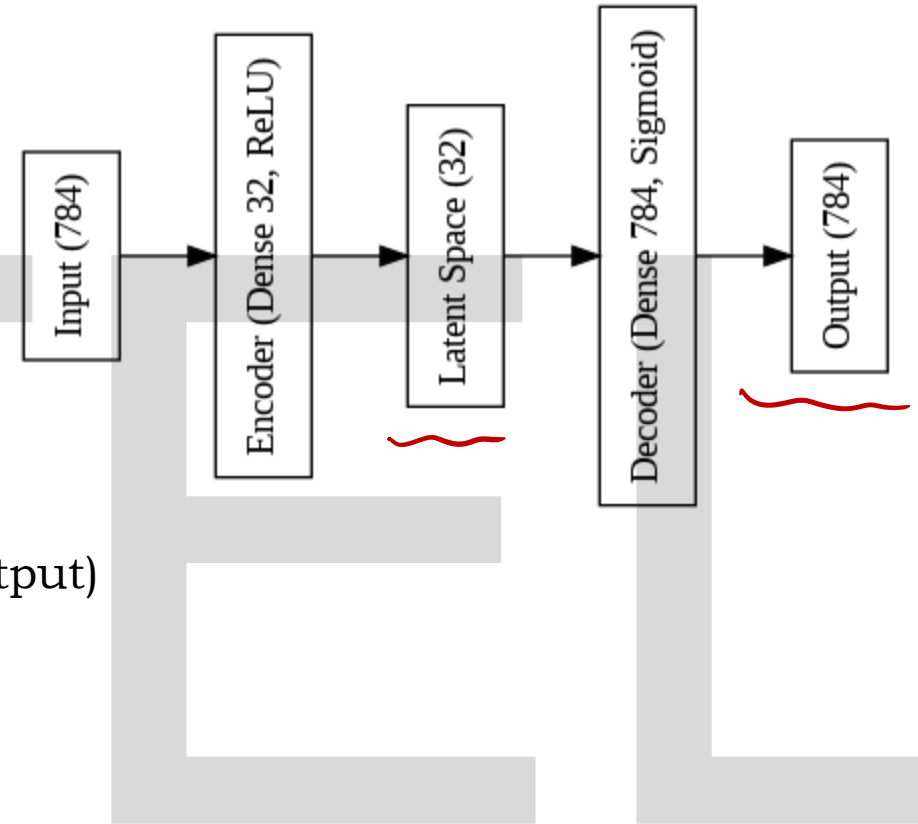
Only ONE hidden layer $\rightarrow$ hence “shallow”



Shallow Autoencoder

```
from tensorflow.keras.layers import Dense, Input
```

```
def simple_autoencoder(inputs):  
    # ----- Encoder -----  
    encoder_output = Dense(32, activation='relu')(inputs)  
  
    # ----- Decoder -----  
    decoder_output = Dense(784, activation='sigmoid')(encoder_output)  
  
    return encoder_output, decoder_output  
  
inputs = Input(shape=(784,))  
latent_output, reconstructed_output = simple_autoencoder(inputs)
```



At the decoder output, the reconstructed values must match the range of the input data.

If preprocessing scales the features to $[0,1]$, we typically use sigmoid activation so the output is bounded in the same range, and binary cross-entropy is commonly used as the loss.

If the input features are unscaled or standardized and can take any real value, we use a linear activation at the output and mean squared error as the loss.

Deep Autoencoder

A **Deep Autoencoder** is an extension of the shallow autoencoder, characterized by:

- **Multiple hidden layers between the input and the output.**
- **Ability to learn hierarchical and abstract representations of data.**

Architecture:

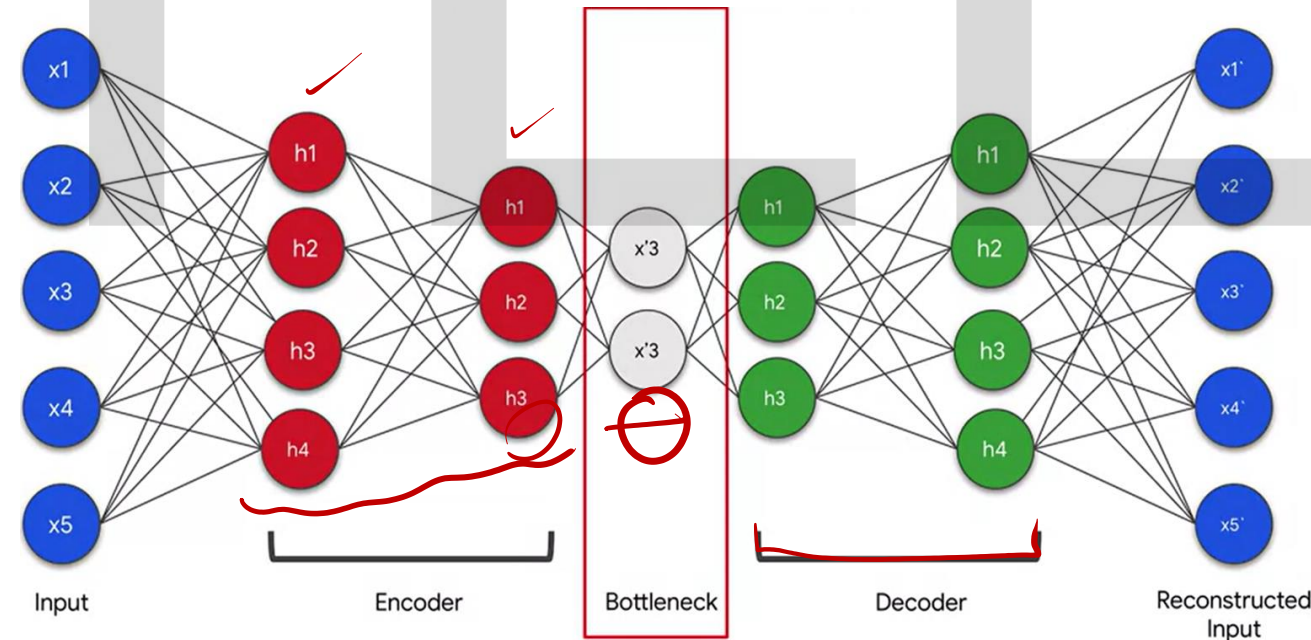
Input Layer → Multiple Encoder Layers → Latent Space → Multiple Decoder Layers → Output Layer

$$X \in R^n \rightarrow h_1 \rightarrow h_2 \rightarrow \dots \rightarrow z \in R^m (m < n) \rightarrow \hat{X} \in R^n$$

Encoder: progressively compresses the input

Latent space: captures high-level features

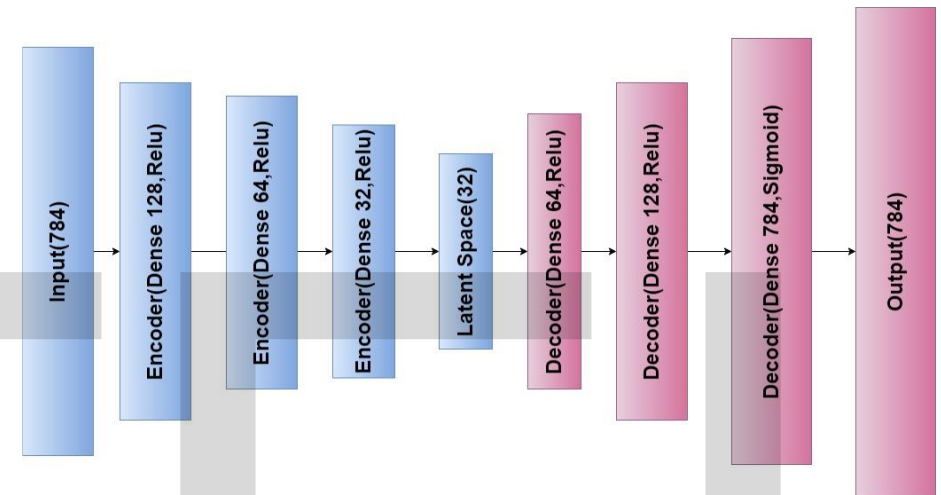
Decoder: progressively reconstructs the input



Deep Autoencoder

```
def deep_autoencoder(inputs):  
    # ----- Encoder -----  
    x = Dense(128, activation='relu')(inputs)  
    x = Dense(64, activation='relu')(x)  
    latent = Dense(32, activation='relu')(x) # Latent Space (32)  
  
    # ----- Decoder -----  
    x = Dense(64, activation='relu')(latent)  
    x = Dense(128, activation='relu')(x)  
    reconstruction = Dense(784, activation='sigmoid')(x)  
  
    return latent, reconstruction
```

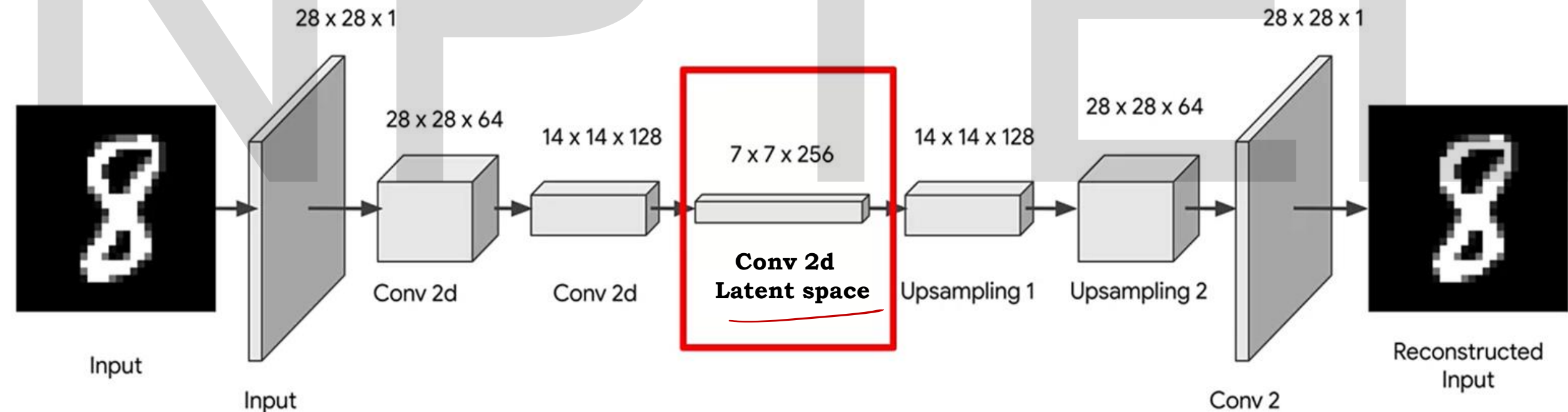
```
inputs = Input(shape=(784,))  
latent_output, reconstructed_output = deep_autoencoder(inputs)
```



Convolutional Autoencoder

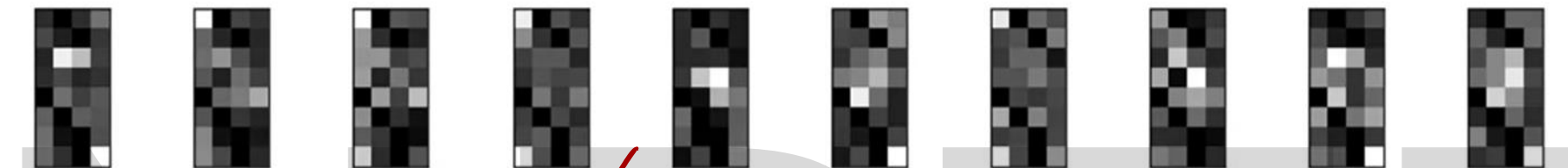
A **Convolutional Autoencoder** is designed for **image data** and uses:

- **Encoder:** Conv + (often) MaxPooling → compresses spatial information.
- **Latent Space Representation:** The output of the final convolutional layer in the encoder serves as the compressed representation (latent space) of the input image.
- **Decoder:** ConvTranspose (or UpSampling + Conv) → reconstructs the image





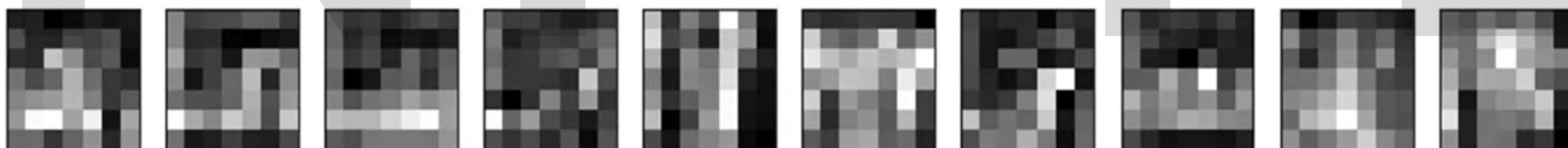
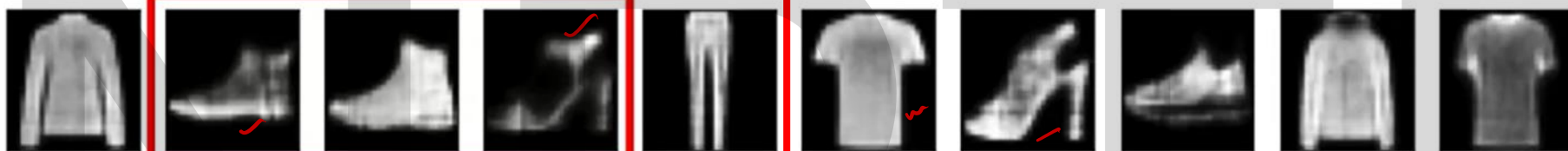
Flattens the image ($28 \times 28 \rightarrow 784$)



DNN

Autoencoder

Fails to preserve spatial relationships like edges, contours, and shapes.

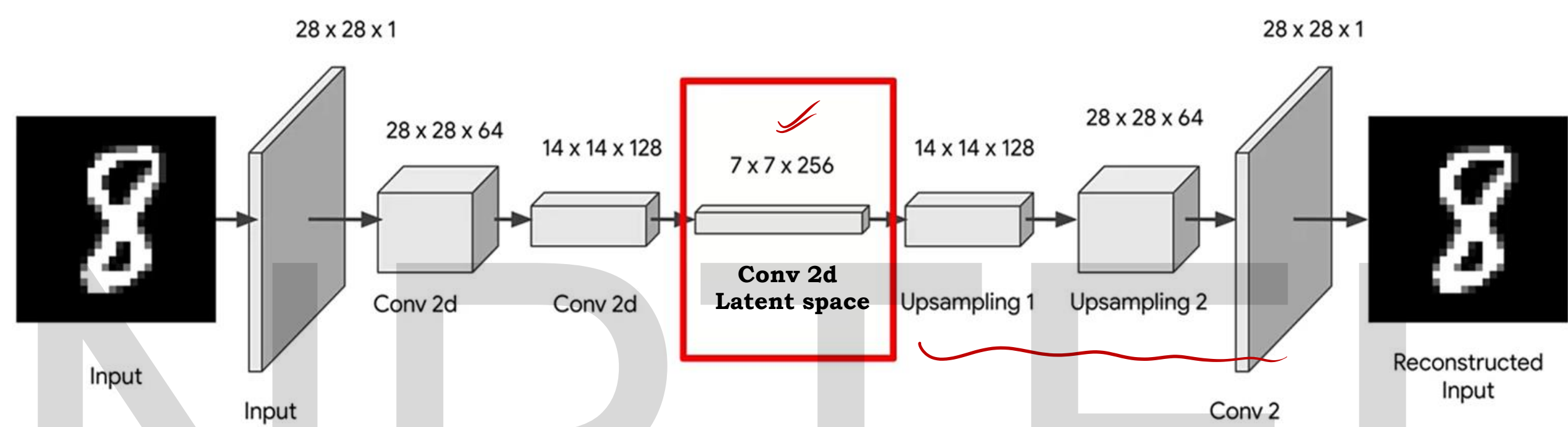


CNN

Autoencoder

- Uses convolutions
- Preserves local spatial structure
- Learns edges, textures, and shapes
- Decoder reconstructs using learned feature maps





```
def conv_autoencoder(inputs):
```

```
# ----- Encoder -----
```

```
# Input: 28x28x1
```

```
enc_conv1 = Conv2D(64, (3,3), activation='relu', padding='same')(inputs)
```

```
enc_pool1 = MaxPooling2D((2,2), padding='same')(enc_conv1)
```

```
# 14x14x64
```

```
enc_conv2 = Conv2D(128, (3,3), activation='relu', padding='same')(enc_pool1)
```

```
enc_pool2 = MaxPooling2D((2,2), padding='same')(enc_conv2)
```

```
# 7x7x128 ← last pooled output
```

```
# ----- Latent Space -----
```

```
latent = Conv2D(256, (3,3), activation='relu', padding='same')(enc_pool2)
```

```
# 7x7x256
```

28x28

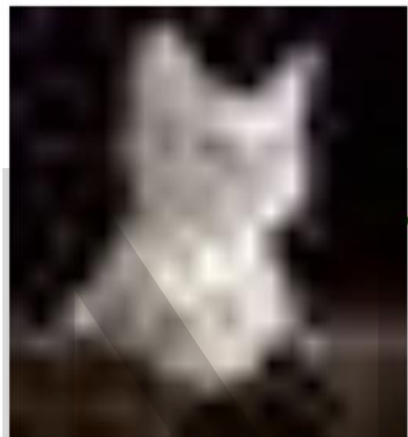
↓

14x14

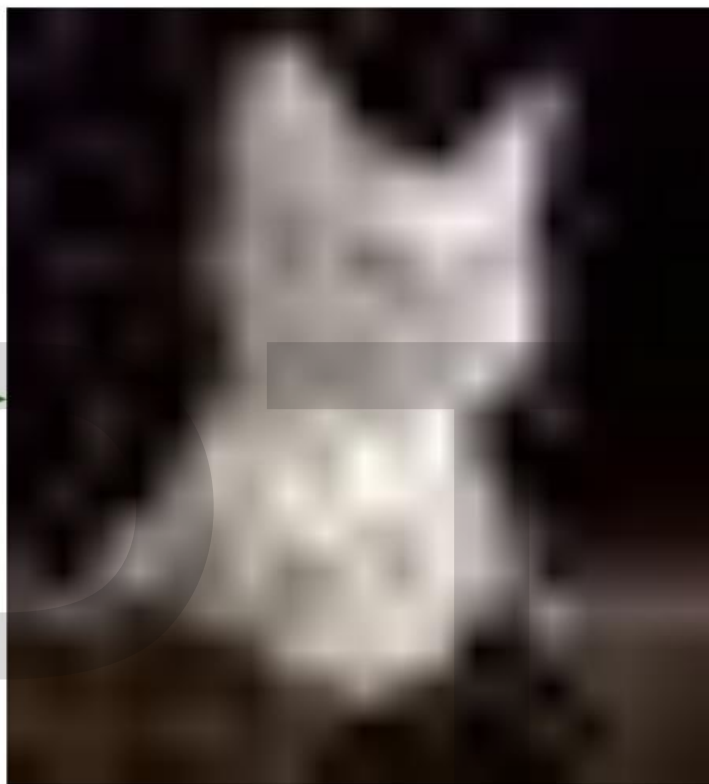
↓

7x7

Upsampling



Upsampling



The feature map produced by the *last conv layer* in the encoder

After **last conv layer**, the feature map size becomes smaller than the original input:

$$28 \times 28 \rightarrow 14 \times 14 \rightarrow 7 \times 7$$

But the **decoder must reconstruct back to 28×28 (input size)**.
So we need a way to **increase spatial resolution** → **Upsampling**.

Increases spatial resolution (e.g., $7 \times 7 \rightarrow 14 \times 14$)

No learnable parameters

$$X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad S = 2$$

Each element expands into a 2×2 block.

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \Rightarrow \begin{bmatrix} \underline{1} & \underline{1} & 2 & 2 \\ \underline{1} & \underline{1} & 2 & 2 \\ \underline{3} & \underline{3} & 4 & 4 \\ \underline{3} & \underline{3} & 4 & 4 \end{bmatrix}$$

Nearest Neighbor Upsampling

Output size = (Input size) $\times$ S (upsampling factor)

Each element is copied into a 2×2 block because the upsampling factor $s = 2$.

$$X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Each element expands into a 2×2 block.

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \Rightarrow \begin{bmatrix} \underline{1} & 0 & 2 & 0 \\ 0 & 0 & 0 & 0 \\ \underline{3} & 0 & 4 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Bed of Nails Upsampling (Zero Insertion)

Transposed convolution operation

$$\text{input} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \quad \text{flipped kernel} = \begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix}$$

$$\text{Transpose Conv} = \text{input} * \text{flipped kernel}$$

$$\begin{aligned} \text{Output size} &= (\text{Input} + \text{kernel} - 1) \times (\text{Input} + \text{kernel} - 1) \\ &= (2 + 2 - 1) \times (2 + 2 - 1) = 3 \times 3 \end{aligned}$$

$$\text{Transpose conv} = G_1 + G_2 + G_3 + G_4$$

$$1 \cdot \begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 4 & 3 & 0 \\ 2 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} G_1$$

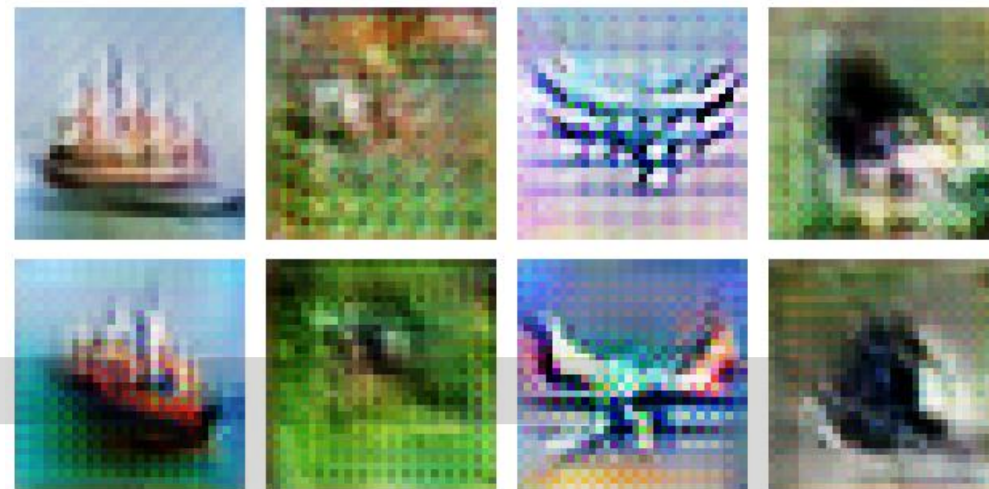
$$\text{Trans Conv} = \begin{bmatrix} 4 & 3 & 0 \\ 2 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 8 & 6 \\ 0 & 4 & 2 \\ 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 \\ 0 & 4 & 3 \\ 0 & 2 & 1 \end{bmatrix}$$

$$2 \cdot \begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 8 & 6 \\ 0 & 4 & 2 \\ 0 & 0 & 0 \end{bmatrix} G_2$$

$$\text{Trans Conv} = \begin{bmatrix} 4 & 11 & 6 \\ 2 & 9 & 5 \\ 0 & 2 & 1 \end{bmatrix}$$

$$0 \cdot \begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} G_3$$

$$1 \cdot \begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 4 & 3 \\ 0 & 2 & 1 \end{bmatrix} G_4$$



Transposed Convolutions has the **checkerboard problem**

(Checkerboard artifacts are unwanted grid-like patterns that appear in images generated using transposed convolutions (deconvolutions)).

They look like: alternating bright/dark pixels, regular square patterns uneven texture, especially in flat regions

This is not noise — it is a systematic artifact caused by the operation itself.

$$\begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \Rightarrow \begin{bmatrix} 1 & 1 & 2 & 2 \\ 1 & 1 & 2 & 2 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} \quad \text{kernel} = \begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix}$$

Upsampling

Convolution

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix} = 1 \cdot 4 + 1 \cdot 3 + 1 \cdot 2 + 1 \cdot 1 = 10$$

$$\begin{bmatrix} 1 & 2 \\ 1 & 2 \end{bmatrix} \cdot \begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix} = 1 \cdot 4 + 2 \cdot 3 + 1 \cdot 2 + 2 \cdot 1 = 4 + 6 + 2 + 2 = 14$$

$$\begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix} \cdot \begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix} = 2 \cdot 4 + 2 \cdot 3 + 2 \cdot 2 + 2 \cdot 1 = 8 + 6 + 4 + 2 = 20$$

$$\begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix} = 1 \cdot 4 + 1 \cdot 3 + 0 \cdot 2 + 0 \cdot 1 = 4 + 3 + 0 + 0 = 7$$

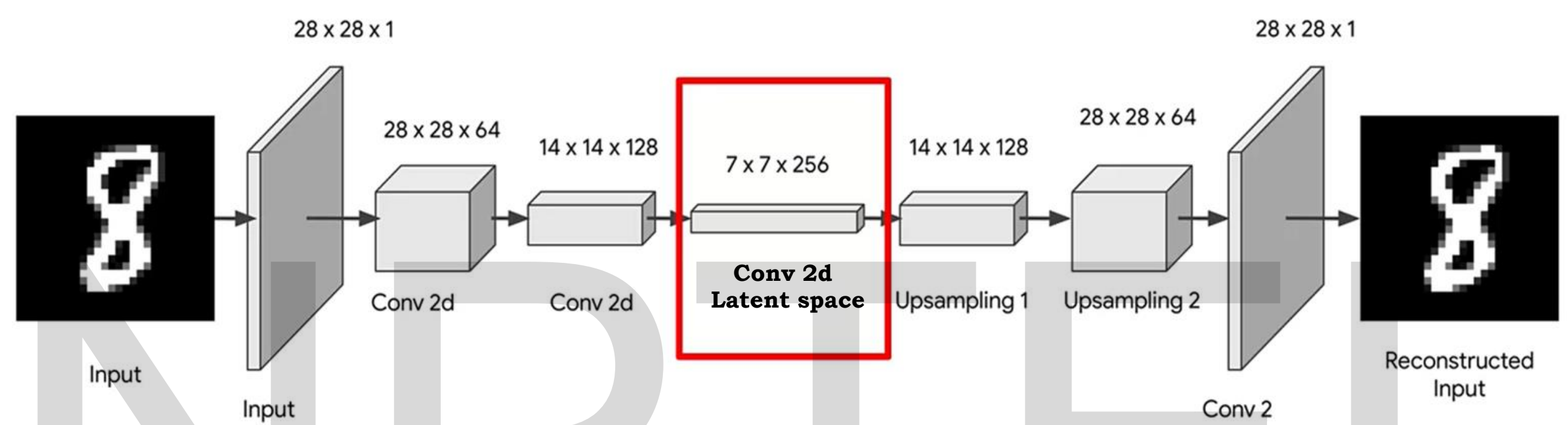
$$\begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix} = 1 \cdot 4 + 2 \cdot 3 + 0 \cdot 2 + 1 \cdot 1 = 4 + 6 + 0 + 1 = 11$$

$$\begin{bmatrix} 2 & 2 \\ 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix} = 2 \cdot 4 + 2 \cdot 3 + 1 \cdot 2 + 1 \cdot 1 = 8 + 6 + 2 + 1 = 17$$

Upsampling + Convolution

$$\text{Conv} = \begin{bmatrix} 10 & 14 & 20 \\ 7 & 11 & 17 \\ 0 & 4 & 10 \end{bmatrix}$$

Upsampling first spreads values uniformly, and the subsequent convolution applies the kernel evenly—avoiding the uneven overlap that causes checkerboard artifacts.



----- Decoder -----

```
dec_up1 = UpSampling2D((2,2))(latent) # 7x7x256 → 14x14x256
```

```
dec_conv1 = Conv2D(128, (3,3), activation='relu', padding='same')(dec_up1)
```

```
dec_up2 = UpSampling2D((2,2))(dec_conv1) # 14x14x128 → 28x28x128
```

```
dec_conv2 = Conv2D(64, (3,3), activation='relu', padding='same')(dec_up2)
```

```
reconstruction = Conv2D(1, (3,3), activation='sigmoid', padding='same')(dec_conv2)
```

```
return bottleneck, reconstruction
```

----- Input -----

```
inputs = Input(shape=(28, 28, 1), name='input_image')
```

----- Outputs -----

```
latent_output, reconstructed_output = conv_autoencoder(inputs)
```

Sigmoid is used at the output layer of an autoencoder to ensure that reconstructed pixel values lie in the valid intensity range [0,1], matching the normalized input image.

Latent space dimensionality controls reconstruction quality ✓



2D latent space



5D latent space



Ground Truth

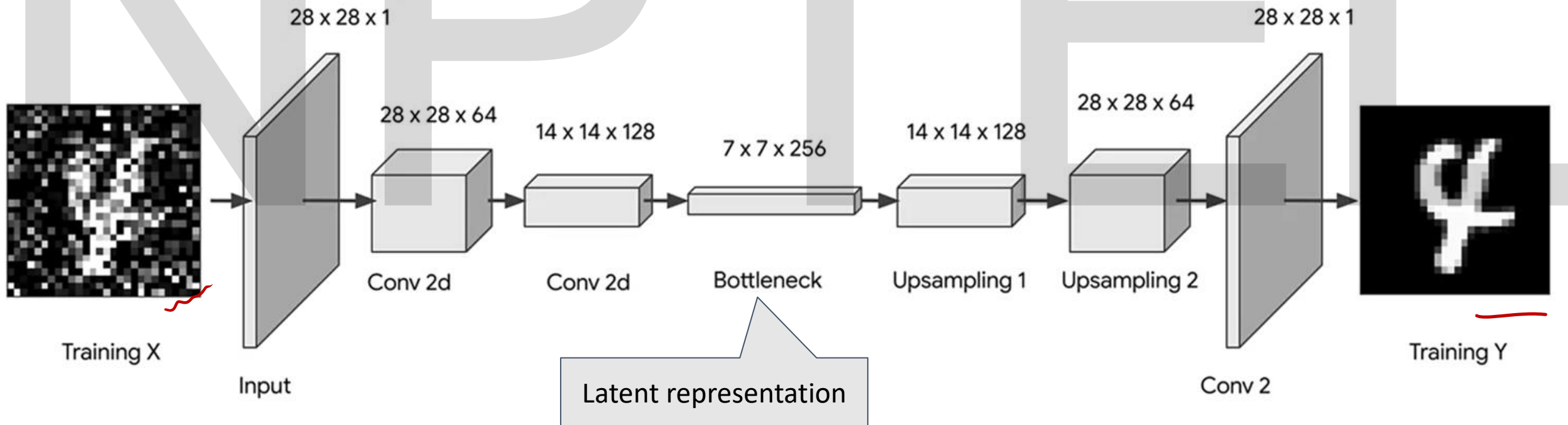


If we choose it too small, the model is forced to discard important details, resulting in poor reconstruction. As we increase the latent dimensionality, the model preserves more information and reconstruction quality improves.

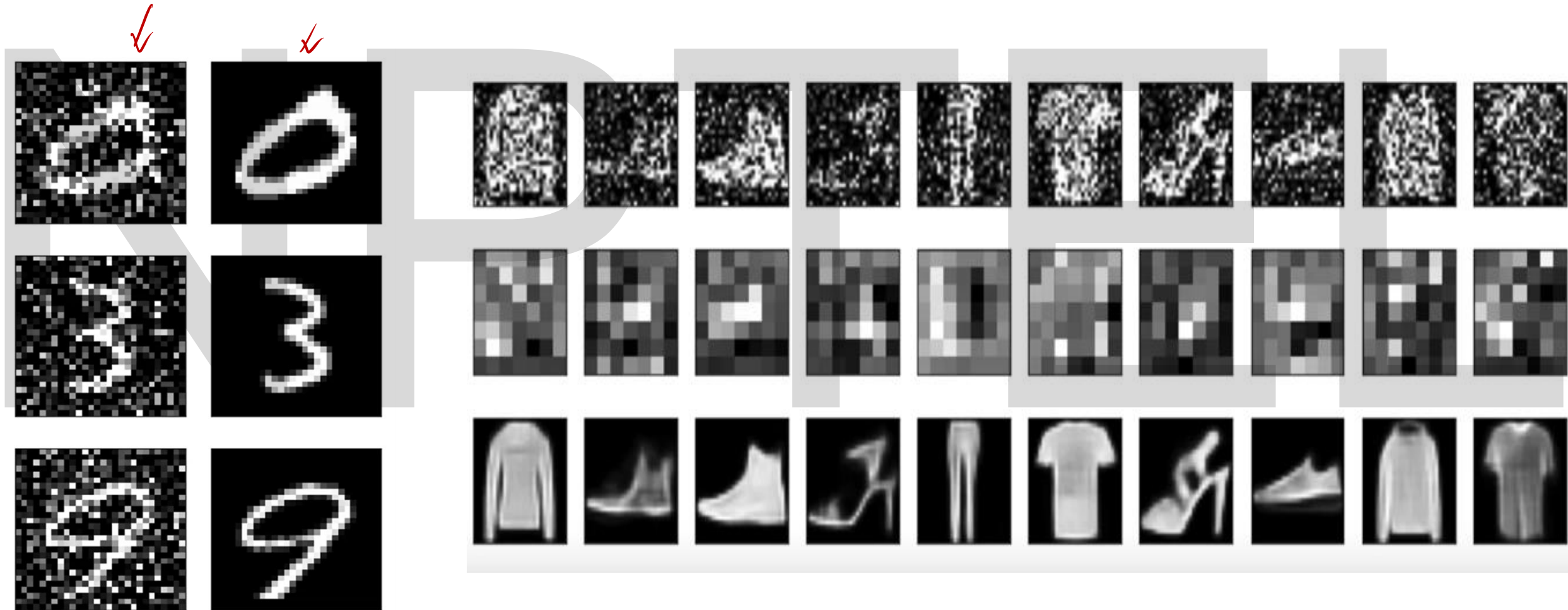
However, beyond a point, increasing latent size gives diminishing returns.

Denoising Autoencoder

- A Denoising Autoencoder (DAE) is a type of autoencoder that is trained to remove noise from its input.
- It is a variant of the traditional autoencoder, but instead of learning to reconstruct the exact input, it learns to reconstruct the original input from a corrupted version of it.



Denoising Autoencoder



Limitations of Autoencoders

1. No guarantee of meaningful latent representation

What happens?

• AEs are trained only to **minimize reconstruction error**

The learned latent features may be optimal for reconstruction but may not be semantically meaningful (**human-interpretable concepts** or **task-relevant properties** (like digit identity, shape, stroke thickness)).

2. Risk of learning identity mapping (overfitting/memorization)

Especially when:

- Latent dimension is large
- No regularization is applied

3. Sensitive to latent space size (hyperparameter tuning)

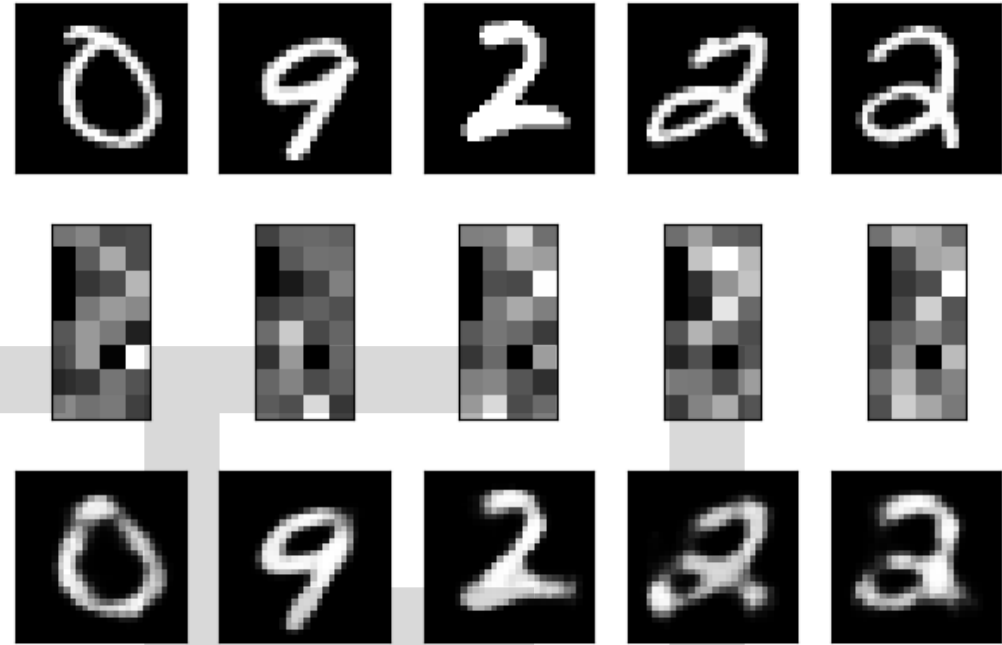
Problem

- Too small latent space → information loss
- Too large latent space → trivial reconstruction (autoencoder learns an almost direct copy of the input, without learning any useful or compact representation).

4. Reconstruction $\neq$ task relevance: Autoencoders are trained only to reconstruct input data, not to optimize performance on downstream tasks such as classification, clustering, or anomaly detection. Therefore, a low reconstruction error does not guarantee that the learned latent representations are useful for these tasks.

5. Basic autoencoders are not true generative models

- The model reconstructs the input using only the information preserved by the encoder.
- Decoder does not recover the original features explicitly.
- It reconstructs an approximation of the input based on the information preserved. Therefore, decoder does not create new information.



Next Session

- Variational Autoencoder
 - Architecture
 - Loss function

NPTTEL

NIPTELL

Thank you